

CERTIFICACIÓN UNIVERSITARIA



UNC



FCEyN



Manual para el  
alumno

Monitoreo



WE WANT  
SKILLS!

**mundosE**

|                                                                                             |          |
|---------------------------------------------------------------------------------------------|----------|
| Manual para el alumno: Monitoreo.....                                                       | 2        |
| <b>Objetivo del encuentro:.....</b>                                                         | <b>2</b> |
| Contenido a desarrollar:.....                                                               | 2        |
| 1. Qué es el monitoreo y por qué es fundamental en la operación de sistemas....             | 3        |
| 2. Monitoreo proactivo y reactivo: diferencias, usos y riesgos.....                         | 4        |
| 3. Monitoreo de infraestructura: servidores, redes, discos y procesos.....                  | 5        |
| 4. Monitoreo de aplicaciones: rendimiento, disponibilidad y experiencia del<br>usuario..... | 6        |
| 5. Nagios: instalación, configuración, servicios y alertas.....                             | 7        |
| 6. Integración entre monitoreo y telemetría: Nagios, Prometheus y Grafana.....              | 8        |
| <b>Bibliografía.....</b>                                                                    | <b>9</b> |

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

## Manual para el alumno: Monitoreo

### Objetivo del encuentro:

Comprender los fundamentos del monitoreo en sistemas y aplicaciones, reconociendo su importancia para garantizar continuidad operativa, anticipar fallas, detectar degradaciones de rendimiento y sostener acuerdos de nivel de servicio. En este encuentro, incorporarás herramientas conceptuales y prácticas para interpretar métricas, diferenciar enfoques de monitoreo, identificar qué debe monitorearse y por qué, configurar soluciones como Nagios, analizar alertas y articular monitoreo tradicional con telemetría moderna.

### Contenido a desarrollar:

- ✓ Definición de monitoreo y su relevancia.
- ✓ Diferencias entre monitoreo proactivo y reactivo.
- ✓ Monitoreo de infraestructura.
- ✓ Monitoreo de aplicaciones y experiencia del usuario.
- ✓ Nagios: funcionamiento, configuración básica y alertas.
- ✓ Integración entre monitoreo y telemetría.

# 1. Qué es el monitoreo y por qué es fundamental en la operación de sistemas

El monitoreo es una disciplina esencial dentro de la administración de sistemas y la ingeniería de *software*. Su propósito es observar de forma continua el estado, el rendimiento y el comportamiento de los componentes de un entorno tecnológico, permitiendo anticipar fallas, garantizar estabilidad y asegurar un desempeño adecuado. Para un estudiante de entornos de producción es fundamental comprender que el monitoreo no es un agregado opcional: forma parte de la estructura que permite que un sistema funcione de manera confiable y bajo control.

En esencia, monitorear significa medir de manera sistemática lo que ocurre dentro de un sistema. Esto incluye desde parámetros básicos como el uso de CPU o memoria hasta métricas complejas como la latencia entre microservicios, el comportamiento de bases de datos, los tiempos de respuesta de una API o la disponibilidad de un servicio crítico. Todas estas mediciones se transforman en información valiosa que permite detectar patrones anómalos, entender el estado real del sistema y tomar decisiones sobre mantenimiento, escalabilidad o correcciones necesarias.

El monitoreo tiene una característica clave: es continuo. No basta con medir el estado cuando surge un problema. Los sistemas modernos demandan observación constante, ya que muchas fallas no se producen de manera súbita, sino como resultado de degradaciones graduales que únicamente pueden detectarse mediante un seguimiento constante. Por ejemplo, un disco rara vez se llena de un día para el otro; generalmente muestra una tendencia de crecimiento que puede anticiparse. Una aplicación no suele volverse repentinamente lenta: a menudo su latencia aumenta progresivamente debido a saturación de recursos o cuellos de botella.

Comprender estas dinámicas es fundamental porque el monitoreo no solo reacciona ante errores, sino que revela comportamientos cotidianos, permitiendo corregir fallas antes de que generen impacto en usuarios o clientes. Un sistema sin monitoreo opera ciegamente y solo advierte un problema cuando ya se produjo un incidente, lo cual afecta el negocio, genera pérdida de tiempo y daña la experiencia del usuario.

Otro aspecto esencial es la relación entre monitoreo y acuerdos de nivel de servicio (SLA/SLO). Ninguna organización puede garantizar disponibilidad o rendimiento sin medirlos. Para evaluar si un sistema cumple con el porcentaje de disponibilidad acordado o si los tiempos de respuesta se mantienen dentro de valores aceptables, es indispensable contar con métricas confiables y continuas. Por eso, el monitoreo es una herramienta estratégica, no solo técnica: permite auditar, justificar decisiones y mantener compromisos de calidad.

Finalmente, el monitoreo es la base de la observabilidad moderna, que combina métricas, logs y trazas para obtener una visión integral del sistema. Aunque en este encuentro nos enfocamos en monitoreo, es importante entenderlo como parte de un ecosistema mayor que permitirá al alumno, en encuentros posteriores, avanzar hacia prácticas más complejas.

## **2. Monitoreo proactivo y reactivo: diferencias, usos y riesgos**

El monitoreo puede aplicarse de dos formas principales: de manera reactiva y de manera proactiva. Comprender esta diferencia es fundamental, ya que determina la capacidad de una organización para anticiparse a problemas o, por el contrario, limitarse a responder una vez que los incidentes ya generaron impacto.

El monitoreo reactivo es aquel que detecta fallas cuando ya están ocurriendo. Se dispara una alerta porque un servidor está caído, una aplicación dejó de responder o un recurso se agotó. Este enfoque es común en entornos con baja madurez operativa, donde el monitoreo se utiliza únicamente como un mecanismo de aviso tardío. Si bien tiene utilidad como último recurso, no evita la interrupción del servicio ni el impacto negativo en los usuarios. Por ejemplo, una API podría dejar de funcionar porque su base de datos no tiene conexiones disponibles; en un enfoque reactivo, la alerta se activaría solo cuando el servicio ya no es accesible, lo cual genera una situación de urgencia.

En contraste, el monitoreo proactivo se orienta a detectar degradaciones antes de que se transformen en incidentes. Este enfoque aprovecha tendencias, umbrales y patrones de comportamiento para anticipar problemas. Lo fundamental aquí es entender que las fallas suelen tener señales previas: uso de CPU que crece lentamente, latencias que aumentan, errores intermitentes, saturación progresiva de memoria o conexiones. El monitoreo proactivo actúa sobre estas señales tempranas y permite solucionar el problema antes de que tenga consecuencias visibles.

Es importante comprender que pasar de un enfoque reactivo a uno proactivo no se trata solo de configurar más alertas; implica establecer métricas relevantes, analizar tendencias, ajustar umbrales y conocer el comportamiento normal del sistema para identificar desviaciones. En la práctica, esto se traduce en intervenciones planificadas: limpiar logs antes de que el disco se llene, reiniciar servicios que presentan fugas de memoria, escalar recursos cuando la carga aumenta o corregir consultas lentas antes de que degraden toda la aplicación.

Un riesgo importante del monitoreo inadecuado es la fatiga de alertas. Cuando un sistema genera demasiadas alertas irrelevantes o confusas, el equipo técnico tiende a ignorarlas. Un buen monitoreo proactivo reduce este riesgo filtrando únicamente las alertas que requieren acción, disminuyendo el ruido operativo.

En resumen, el monitoreo proactivo permite mantener sistemas estables, evitar interrupciones, planificar capacidad y sostener decisiones operativas con datos confiables, mientras que el monitoreo reactivo solo sirve como mecanismo tardío. Ambos modelos conviven, pero el objetivo es que la mayor parte del monitoreo sea proactivo.

### **3. Monitoreo de infraestructura: servidores, redes, discos y procesos**

El monitoreo de infraestructura es el nivel más tradicional y constituye el cimiento de cualquier estrategia de observación de sistemas. Se enfoca en los componentes físicos y virtuales que soportan la operación de servicios y aplicaciones. Comprender qué métricas son críticas y cómo interpretarlas es esencial para mantener un entorno estable.

Uno de los principales recursos a monitorear es la CPU. Esta métrica indica la capacidad de procesamiento utilizada por un servidor y permite identificar saturaciones, procesos mal optimizados o tareas que consumen recursos de manera inesperada. Una CPU constantemente por encima del 90% puede indicar que el sistema está al límite, lo cual afecta directamente el rendimiento de aplicaciones alojadas.

El monitoreo de memoria (RAM) también es central. Cuando la memoria disponible se reduce de forma sostenida, los sistemas pueden comenzar a utilizar swap, lo cual degrada notablemente el rendimiento. Además, pérdidas de memoria en aplicaciones generan saturaciones progresivas que pueden detectarse mediante el monitoreo adecuado.

El uso de disco es un elemento crítico que suele ser fuente de incidentes evitables. Un disco lleno no solo impide guardar datos: puede detener servicios, provocar corrupción de archivos y afectar procesos del sistema operativo. Por eso es esencial monitorear la disponibilidad de espacio, la tasa de crecimiento del disco y la capacidad de entrada/salida (IOPS). La tasa de escritura y lectura también contribuye a identificar cuellos de botella.

En cuanto a la red, es necesario monitorear la latencia, el ancho de banda y los errores en las interfaces. Problemas de red pueden manifestarse como lentitud en aplicaciones, interrupciones en servicios distribuidos o fallas en la comunicación entre microservicios. Detectar patrones de congestión o pérdida de paquetes permite resolver problemas antes de que escalen.

El monitoreo de procesos es clave para identificar comportamientos anómalos en servicios internos. Un proceso que consume demasiado CPU, uno que deja de existir, o uno que se reinicia constantemente son señales críticas de inestabilidad.

Por último, el monitoreo de infraestructura permite generar una línea de base o "baseline": valores normales de recursos bajo carga típica. Esto facilita identificar comportamientos extraños que, aunque no superen umbrales, indican un problema incipiente.

## 4. Monitoreo de aplicaciones: rendimiento, disponibilidad y experiencia del usuario

El monitoreo de aplicaciones permite evaluar cómo funciona realmente un sistema desde la perspectiva del usuario y del negocio. A diferencia del monitoreo de infraestructura, que muestra el estado técnico del servidor, el monitoreo de aplicaciones revela si los servicios están cumpliendo con su propósito funcional.

Una métrica esencial es el tiempo de respuesta. Este valor indica cuánto tarda el sistema en procesar una solicitud. Tiempo de respuesta elevado afecta directamente la experiencia del usuario. Aunque la infraestructura esté estable, una API podría responder lentamente debido a consultas ineficientes en la base de datos, saturación interna o problemas de diseño.

Otra métrica importante es la tasa de errores. Los errores 4xx y 5xx muestran fallas que impiden el funcionamiento correcto de un servicio. Por ejemplo, un aumento en errores 500 es una señal clara de falla interna que debe investigarse de inmediato.

El *throughput*, o cantidad de solicitudes procesadas por unidad de tiempo, refleja la capacidad del sistema para manejar carga. Un descenso repentino puede indicar una degradación silenciosa.

El monitoreo de integraciones externas también es fundamental. Muchos sistemas dependen de APIs externas cuyo desempeño afecta directamente a la aplicación principal. Si una API de un proveedor presenta latencia, esto puede repercutir en tiempos de respuesta del sistema completo.

La experiencia del usuario puede monitorearse con dos técnicas principales:

- **RUM (Real User Monitoring):** captura datos reales desde navegadores o aplicaciones.
- **Monitoreo sintético:** simula accesos para evaluar disponibilidad desde distintos puntos de red.

Estas técnicas permiten identificar fallas que no aparecen en el monitoreo interno, como problemas de DNS, latencias regionales o degradaciones que solo afectan a ciertos segmentos de usuarios.

Finalmente, correlacionar métricas de aplicación permite obtener una visión amplia del funcionamiento. Por ejemplo:



- Tiempo de respuesta alto + CPU baja → problema lógico.
- Errores 500 + saturación de base de datos → cuello de botella.
- Caída del *throughput* + normalidad en infraestructura → problemas de aplicación.

## 5. Nagios: instalación, configuración, servicios y alertas

Nagios es una herramienta clásica y ampliamente utilizada para monitoreo basado en chequeos activos. Su arquitectura se centra en un servidor central que ejecuta verificaciones periódicas mediante *plugins* especializados.

Es importante comprender su funcionamiento permite sentar bases sólidas antes de avanzar hacia herramientas más modernas. Nagios se organiza mediante definiciones declarativas: los hosts, servicios, contactos y comandos se configuran en archivos de texto estructurados. Esta simpleza hace que su uso sea didáctico y accesible.

Los *plugins* son el corazón del sistema. Existen *plugins* para verificar conectividad (*check\_ping*), disponibilidad HTTP (*check\_http*), uso de disco (*check\_disk*), carga del sistema (*check\_load*) o procesos específicos. Además, mediante agentes como NRPE es posible ejecutar chequeos internos dentro de servidores remotos.

Las alertas en Nagios se definen mediante umbrales. Por ejemplo, un servicio puede pasar a WARNING si supera el 70 % de uso y a CRITICAL si supera el 90 %. Estos umbrales deben configurarse con cuidado para evitar tanto falsos positivos como alertas tardías.

El alumno debe comprender que Nagios permite gestionar contactos, grupos y horarios de notificación. Esto garantiza que las alertas lleguen a las personas correctas en el momento adecuado. También permite definir ventanas de mantenimiento para evitar alertas durante trabajos planificados.

La gestión del estado incluye chequeos blandos (*soft*) y duros (*hard*). Un estado blando indica que el problema podría ser temporal, mientras que uno duro confirma que la situación persiste y requiere notificación. Esta lógica reduce alertas erróneas y mejora la confiabilidad del sistema.

Aunque Nagios no es una herramienta de telemetría moderna, su capacidad para detectar estados críticos lo convierte en un componente fundamental de cualquier esquema de monitoreo.



## 6. Integración entre monitoreo y telemetría: Nagios, Prometheus y Grafana

El monitoreo tradicional y la telemetría no compiten: se complementan. Nagios ofrece chequeos puntuales y alertas confiables para detectar fallas específicas. Prometheus, en cambio, recolecta métricas en forma de series temporales y permite analizar tendencias. Grafana visualiza esas métricas de forma clara y correlacionada.

Integrar estas herramientas permite obtener una visión completa del sistema. Por ejemplo:

- Nagios alerta que un servidor está lento.
- Prometheus muestra que la memoria crecía desde hace horas.
- Grafana revela que coincide con un aumento en consultas a la base de datos.

La telemetría permite ver comportamientos históricos, identificar patrones y diagnosticar causas raíz. Nagios permite actuar rápidamente ante estados críticos. Esta combinación hace que el monitoreo sea preventivo y reactivo a la vez.

Prometheus se basa en el concepto de scraping: en lugar de recibir métricas, las consulta activamente desde endpoints que exponen datos. Esto permite métricas en alta frecuencia y bajo costo. Grafana permite combinarlas con paneles visuales, alertas avanzadas y correlación temporal.

Comprender esta integración es clave: los sistemas modernos rara vez dependen de una sola herramienta. El monitoreo completo surge de unir chequeos, métricas, logs y trazas en un ecosistema coherente.

# Bibliografía

Bartholomew, J. (2023). *Modern Monitoring Strategies*. O'Reilly Media.

Google SRE Team. (2023). *Site Reliability Engineering*. O'Reilly Media.

Nagios Enterprises. (2024). *Nagios Core Documentation*.

<https://www.nagios.org/documentation/>

Prometheus Authors. (2025). *Prometheus Documentation*.

<https://prometheus.io/docs/>

Grafana Labs. (2025). *Grafana Documentation*. <https://grafana.com/docs/>

Rouse, M. (2024). *Infrastructure Monitoring Fundamentals*. TechTarget.